

Module : Kotlin (M202)

TP N° 22

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVOAM101 - DEVOAM102 Niveau : 2ème année

Exercice 1 :

Classes et héritage :

- Crée une classe Personne avec des propriétés nom et âge, et une méthode parler().
- Crée une classe dérivée Étudiant qui ajoute une propriété classe.
- Redéfinis la méthode parler().

NullSafety :

- Crée une fonction qui prend une chaîne de caractères nullable et renvoie sa longueur, ou 0 si la chaîne est null.

Interfaces :

- Crée une interface Conducteur avec une méthode conduire(). Implémente cette interface dans les classes Voiture et Moto.

Objets Kotlin :

- Crée un objet singleton GestionnaireDeConnexion avec une méthode connecter() et une propriété pour stocker l'état de la connexion.

Sealed class :

- Crée une classe scellée Résultat avec deux sous-classes : Succès et Erreur. Implémente une fonction qui retourne un Résultat.

Data class :

- Crée une data class Livre avec des propriétés titre et auteur, et utilise les méthodes générées automatiquement comme toString() et copy().

Nested class :

- Crée une classe Ordinateur avec une classe imbriquée Processeur. La classe Processeur devrait avoir une méthode pour retourner les spécifications du processeur.

Enum :

- Crée un enum Saison avec les valeurs Été, Automne, Hiver, et Printemps. Ajoute une propriété temperatureMoyenne à chaque valeur.

Extensions :

- Ajoute une fonction d'extension à la classe String pour vérifier si elle est un palindrome.

Collections et ranges :

- Utilise une collection List d'entiers et filtre les nombres pairs. Utilise un range pour afficher les nombres de 1 à 10.

Expressions et opérateurs :

- Crée une fonction qui utilise l'opérateur ternaire (?:) pour renvoyer une valeur par défaut si un objet est null.

Appels asynchrones :

- Utilise async et await pour lancer deux tâches asynchrones, puis retourne la somme des résultats.

Lazy loading :

- Crée une classe Utilisateur avec une propriété profilCompleet qui est initialisée paresseusement.

Lambda expression :

- Crée une liste de nombres et utilise une expression lambda pour trouver la somme des carrés des nombres pairs.

Gestion des exceptions :

- Crée une fonction qui divise deux nombres et gère l'exception ArithmeticException si le diviseur est zéro.

High-order Functions :

- Crée une fonction appliquer qui prend une liste et une fonction en paramètre, et applique cette fonction à chaque élément de la liste.

Fonctions inline :

- Crée une fonction inline qui prend une fonction lambda et l'exécute.

Réflexion :

- Utilise la réflexion pour inspecter les propriétés et méthodes d'une classe Personne.

Type safe builders :

- Crée une fonction de type safe builder pour construire un objet Html avec des balises comme div, span, etc.

Type Aliases :

- Crée un alias de type pour une fonction qui prend deux Int et retourne un Int.

Coroutines Scope :

- Crée une coroutine dans un CoroutineScope personnalisé pour exécuter une tâche en parallèle.

Fonction launch :

- Utilise launch pour exécuter une tâche asynchrone qui affiche des messages toutes les secondes pendant 5 secondes.

Fonction await :

- Crée une fonction asynchrone qui utilise await pour récupérer des données après un délai simulé.

Jobs :

- Crée une coroutine avec un Job, annule-la après un certain temps et vérifie son statut.

Dokka :

- Ajoute des commentaires KDoc à une classe, puis génère la documentation avec Dokka.

KDoc :

- Utilise KDoc pour documenter une fonction additionner qui prend deux Int et retourne leur somme.

Kapt :

- Implémente une simple annotation avec Kapt pour générer du code lors de la compilation.

Gradle :

- Ajoute une tâche personnalisée à un projet Gradle qui affiche un message lors de l'exécution.

Notions de l'injection des dépendances :

- Implémente un simple exemple d'injection de dépendances sans framework avec des constructeurs.

Utilisation de Hilt :

- Implémente Hilt pour injecter une dépendance dans une classe Repository et une activité Android.